claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_49aab888-e10\agent-a358fd3dad3e51485.jsonl`
- SHA-256: `253bf88a30c7d9a2d09cfe3971c3215d4296ae6f9fc9bf989ef70becdf88b207`
- Source modified: `2026-06-08T13:09:34+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_49aab888-e10`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`

Transcript

1. user (2026-06-08T13:04:39.584Z)

DIMENSION = LABEL & DATA REUSE / DATA ENGINEERING. Use WebSearch + WebFetch where useful. Given we have human rally [start,end] CSVs + ending_reason + WASB trajectories for a handful of matches, how do we maximize their value for training across model generations? Cover: (a) converting segment labels into the formats the TAL/segmentation models want; (b) amplifying few labels ? self-training / teacher-student / pseudo-labeling (WITHOUT using paid-LLM outputs as training data), temporal data augmentation, active learning to prioritize what to tag next; (c) how many labeled matches are realistically needed for each model class; (d) a reusable, model-agnostic 'labeled-corpus' format so the same labels feed heuristics, TAL, and Option-B. Cite sources for the data-efficiency claims.

Keep it aligned to OUR situation:
OUR SITUATION (a local-first racket-sports highlight indexer + rally detector):
- GOAL: an OWNED, on-device-capable model for RALLY TEMPORAL SEGMENTATION (predict rally [start,end] intervals in a match video), evolving toward "Option B" = an OWNED END-TO-END MULTIMODAL model (frames + audio + pose -> rallies/highlights) fine-tuned on our own data. Privacy lever = local inference, user video never leaves the device.
- HARD GUARDRAILS (non-negotiable): (1) base/teacher models for TRAINING must be OPEN Apache-2.0 (or MIT/BSD) ONLY; video-capable bases like Qwen-VL / Gemma family are candidates BUT **Gemma 3 is REJECTED (viral/non-standard license)**. (2) NEVER use paid LLM outputs (Gemini/OpenAI/Anthropic) as TRAINING data ? they are inference/eval/teacher-for-eval only (terms + copyright). (3) Exclude minors from the training pool; biometrics/gait = future (GDPR Art. 9). (4) Generic data schema; coach->athlete is an optional overlay.
- DATA ASSETS WE WANT TO REUSE FOR TRAINING: human-labeled rally [start,end] CSVs per match (currently Adarsh-and-Avi 96 rallies, Kushagra 32, more coming; schema rally_number,start_time,end_time,ending_reason,sport) + per-rally ending_reason labels (winner/forced_error/unforced_error/service_fault/let) + per-video WASB shuttle-trajectory CSVs (Frame,Visibility,X,Y; from a frozen MIT-licensed shuttle detector) + (eval-only, NOT training) Gemini silver labels. Footage = single-camera amateur GoPro, 1080p/4K, 60fps.
- CURRENT PIPELINE: WASB (frozen vision shuttle detector) -> trajectory -> hand-tuned windowing + a net-crossing "gate" + a small distilled LogisticRegression on trajectory features. Honest results: threshold tuning ceilings ~F1 0.73 vs HUMAN labels on one match; a learned model UNDERPERFORMS baseline at n=2 videos. Bottleneck = more golden data + WASB recall, not the method.
- CONSTRAINTS: solo founder, BANGALORE-based (cost-sensitive; cloud GPU pricing + data-residency matter), local hardware = ONE RTX 2070 Super (8GB) + WSL2. User explicitly mentioned Vertex AI but wants MORE options + a detailed comparison.
- THE TASK CLASS is temporal action localization/segmentation (action = "rally"); labels are segment intervals.

Grounding brief:
{
  "owned_model_goal": "Build an OWNED, on-device-capable generative video-QA model (fine-tuned from an Apache-2.0 video VLM base) for RALLY TEMPORAL SEGMENTATION and future end-to-end multimodal sport understanding. Privacy lever: local inference, user video never leaves device.

The model plugs into the existing provider registry with paid Gemini as an always-ready fallback. Long-term path (Option B): collapse modular local cues (WASB trajectory, audio hit detection, pose detection) into a single fine-tuned end-to-end model. Near-term (Option A, already shipping): modular signal fusion on frozen or near-frozen bases. The moat is NOT the model weights ? it's the consented badminton-specific video?QA dataset + the eval harness. Owned model becomes primary only after scaffolding (async jobs, multi-tenant, storage/compute abstractions) is rock-solid and the golden-set eval/training corpus is healthy.",
  "constraints": [
    "Base model MUST be Apache-2.0 clean (commercial-viable, fine-tunable, derivative-wrappable). REJECT Gemma 3 (viral Google PUP), Llama Vision (700M-MAU cap + naming mandate). APPROVED bases: Qwen2.5-VL-7B/32B (Apache-2.0 at those sizes only; 3B/72B custom-licensed), Qwen3-VL-4B/8B, Gemma 4 multimodal (Apache-2.0, 2026), InternVL2.5 (MIT).",
    "HARD RULE: Training data must be Apache-2.0 or MIT open-model outputs ONLY. FORBIDDEN: any outputs from paid frontier APIs (Gemini, OpenAI, Anthropic, Claude) as training signal ? they are competing-model clauses and breach terms. Paid APIs are inference/evaluation/teacher-reference only, never training data sources. This is non-negotiable per legal.",
    "Exclude minors' video from the training pool BY DEFAULT. Training requires separate, explicit opt-in consent (distinct from processing consent). Per-user training adapters trained only on that user's own consented video; deletion = drop the adapter.",
    "Generic data schema (no coach-athlete hardcoding). Coach/athlete overlay is optional, orthogonal. Data model stays reusable for solo users, academies, power users.",
    "Training evaluation must use a held-out golden QA eval set (controlled, curated, not user-contributed). Agent-runnable training requires trustworthy pass/fail verdict (not eyeballing). Regression testing locked to golden labels to prevent metric inflation from data leakage.",
    "Video-QA fine-tuning works ONLY on labeled data. Currently available: 96 (Adarsh+Avi) + 32 (Kushagra) human-labeled rally [start,end] CSVs per match + Gemini silver labels (eval-only, not training). 5k?20k curated clip?QA pairs is the realistic training band."
  ],
  "data_assets": [
    "Human-labeled rally [start,end] CSVs per match (Adarsh-and-Avi 96 rallies, Kushagra 32; schema: rally_number, start_time, end_time, ending_reason, sport). Source: sports-data-collector repo, curate export with commercial-license gate.",
    "Per-rally ending_reason labels (winner, forced_error, unforced_error, service_fault, let) ? rally semantics feeding the QA-pair dataset.",
    "Per-video WASB shuttle-trajectory CSVs (Frame, Visibility, X, Y) from frozen MIT-licensed WASB-SBDT shuttle detector. Trajectory is a primary local signal for multi-cue fusion.",
    "Gemini silver labels (eval and teacher-reference ONLY, explicitly not training-data). Used to measure oracle quality and as a training-time prior (Option A modular fusion), never harvested as outputs to train the owned model.",
    "Single-camera amateur GoPro footage: 1080p/4K, 60fps. 22/22 ShuttleSet matches re-acquired at best available resolution (Phase 2 complete). ~26 GB corpus licensed/owned, no user uploads.",
    "Golden-set ground-truth expansion via Roora vendor (Bengaluru, initial pilot underway): 3-sport pilot (badminton, pickleball, table tennis) with validated rally CSV schema and annotation guideline.",
    "Sibling rallyannotator repo (MIT, open-source, VLC-based): owner can self-produce expert-labeled eval clips. Feed via `import-local` to the collector and `--labels` loaders to the indexer."
  ],
  "task_framing": "**Task: design a training study blueprint for the OWNED video-QA model that clarifies (1) what data is safe/available now, (2) which base model to lock, (3) which fine-tuning backend fits our constraints, and (4) how to stage the transition from heuristics ? Option A modular fusion ? Option B end-to-end owned model.**\\n\\n**Current pipeline (heuristics + frozen detectors):** WASB (frozen vision) ? trajectory ? hand-tuned windowing + net-crossing gate + distilled LogisticRegression on trajectory features. Honest baseline: threshold-tuned F1 ? 0.73 vs human labels (one match); learned model underperforms at n=2 videos. Bottleneck = golden data + WASB recall, not method.\\n\\n**Option A (now ? 6?12 months):** Modular, fully-local cues: WASB trajectory + audio hit-detection (E1 probe AMBER / leaning negative, gated on more golden labels) + pose/court serve-gating (validated but heavy). Each cue swappable, independently testable, generates training signal for the end-to-end model. Fusion happens in our rally layer, not inside frozen detectors.\\n\\n**Option B (12?24+ months, after scaffolding + healthy corpus):** Collapse modular cues into a single owned multimodal rally model fine-tuned on (frames + audio + pose/stance + ?) ? rally [start, end]. Removes recall ceiling imposed by WASB candidate windows and licensing question on academic-trained weights. Requires: (a) enough golden eval/training video (owner self-labels via rally-annotator + vendor labels), (b) end-to-

end training harness (nanochat-style operability: one command, report card, deterministic, reproducible), (c) a measured improvement bar vs. the modular Option A.\\n\\n**Bridge data flow (cross-repo, ADR-002 contract):** sports-data-collector's per-match rally CSVs + manifest.json (commercial-license gate) ? indexer's golden eval set and training signal. Collector's 5-sport tables (badminton/tennis/cricket/pickleball/table-tennis) share a (video_id, ordinal, start_time, end_time) shape, so the bridge is sport-generic. Golden-set expansion funded via Roora (Bengaluru) ? initial 3-sport pilot.\\n\\n**Immediate blockers on training-study readiness:** (1) **Owned-model base selection + licensing lock** (Qwen vs Gemma 4; verify per-checkpoint commercial terms). (2) **Fine-tuning backend decision** (self-managed ms-swift on rented GPU [RunPod/Lambda/Modal] vs. managed Together/Fireworks vs. enterprise Predibase). (3) **Golden QA eval harness design** (the crux: nanochat-style pass/fail verdict). (4) **Training data schema & consent** (how to convert rally CSVs + ending-reason labels + Gemini reference ? clip?QA pairs without using paid outputs; minors exclusion). (5) **Data leakage guardrails** (`train | eval` split by match, not clip; overlap detection). (6) **Training job orchestration** (how it slots into the async job model P0, runs on ComputeBackend, fits BYO-compute).\\n\\n**Success criteria for the study:** (a) Blueprint for converting existing golden labels into 5k?20k curated clip?QA pairs. (b) Confirmed Apache-2.0 base + verified commercial DPA from the fine-tune backend. (c) Proof-of-concept golden QA eval gate that blocks a regression-quality model. (d) Staged roadmap: Option A (modular, 2?4 weeks) ? Option B e2e model (after corpus health signals, 3?6 months post-scaffold). (e) Clear cost/latency comparison (self-hosted 7B VLM vs. Gemini per-call on target workload).\"",

  "roadmap_alignment_notes": [

    "**Sequencing principle (owner directive):** Make scaffolding (P0?P4 in PLATFORM_ARCHITECTURE §6) rock-solid FIRST, keep the known paid stack (Gemini) as an always-ready fallback, and only then shift focus to owned-model quality (P5). This study unblocks P5's design but does NOT start implementation until P0?P4 infrastructure is delivered.",

    "**P0 (async job model, single-tenant)** is the prerequisite: owned-model fine-tuning runs as an orchestration job on ComputeBackend GPUs (§5A / DEFERRED #16). Without async + job dispatch, training is not operationalizable.",

    "**P1?P3 (multi-tenant, storage/compute abstraction)** enable the data flywheel: training corpus lives in Storage (§3.1), training/inference jobs dispatch to ComputeBackend (§3.2), per-user adapters live in the compute tier. Without these, the owned model is stranded on a single box.",

    "**P4 (rich rally/shot semantic metadata)** feeds P5: the annotation registry + candidate_segments + golden-set labels are the training-signal source. Per-rally ending_reason labels + human-correction loop ? QA-pair generation.",

    "**C14 (COMMERCIALIZATION.md)** tracks the owned-model licensing + training-data policy as a P1 blocker: Apache-2.0 base lock, open-model teachers only, no paid-LLM training signal, separate explicit training opt-in, minors excluded. This study must resolve C14 before any code lands.",

    "**ADR-004 substrate decision** (2026-06-02 update): Phase 4 offline segmenter is WASB shuttle-motion, not YOLO player-motion (YOLO @ IoU-0.5 recall too low). The owned end-to-end model (Option B) will eventually subsume WASB's role. ADR-007 audio fusion is the Option A stepping stone.",

    "**ADR-007 direction (accepted):** Audio hit-detection is the next rally-detection cue (Option A). It is fully local, permissive, and fuses cleanly with the distilled classifier. Gated on golden labels (E1 probe AMBER, awaits more data). This is the realistic 2?4 week Option A milestone while waiting for scaffold + corpus growth.",

    "**Golden-set expansion (ADR-006 / Roora pilot):** Vendor-produced labels are the highest-leverage move. With Roora on 3-sport badminton/pickleball/table-tennis, the training corpus grows month-on-month. The training study should plan for incremental re-training (CI-integrated, agent-runnable) as each batch lands.",

    "**Long-term moat (§5A, PLATFORM):** The defensible assets are (a) the consented, badminton-specific video?QA dataset and (b) the eval harness. Do NOT message 'we own an LLM' ? the base VLM (Qwen/Gemma) is freely downloadable. Message the dataset + eval precision instead.",

    "**Cost & margin (C7 / COMMERCIALIZATION):** Self-hosted 7B VLM (?16 GB VRAM) on rented GPU is cheaper per inference than Gemini only if: (a) sustained QA volume is high, (b) training corpus is defensibly large, (c) per-call API cost pain is real. Below those thresholds, paid API is cheaper than standing GPU + second stack. Benchmark before committing budget.",

    "**Training harness operability (§5A, PLATFORM):** The goal is 'an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled video and get a trustworthy ship/no-ship verdict, unattended.' This requires: (1) one command, end-to-end (ms-swift engine, recipe-driven); (2) one complexity dial (auto-derived defaults); (3) automatic report card (held-out golden QA eval vs target); (4) reproducible + deterministic + cheap; (5) minimal/readable. Nanochat is the reference for operability (not the engine). The harness = thin glue over ms-swift + the golden

```
eval + the provider registry."
  ]
}
```

2. assistant (2026-06-08T13:04:42.495Z)

```
I'll research this thoroughly. Let me start by examining our existing codebase to ground the
answer in what we actually have, then do targeted web research on data-efficiency claims.
```

3. user (2026-06-08T13:04:43.530Z)

```
docs\EVALUATION.md
docs\TRACKNET_WSL_SETUP.md
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\README.md
docs\CODE_MAP.md
docs\DEFERRED_HARDENING_PLAN.md
docs\PMF_MARKET_RESEARCH.md
docs\ROADMAP.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\decisions\DECISIONS.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\past_projects\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\COMMERCIALIZATION.md
docs\PLATFORM_ARCHITECTURE.md
docs\BACKLOG.md
docs\GOLDEN_SET_VENDORING.md
docs\README.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\archives\quality-iterations\README.md
```

4. assistant (2026-06-08T13:04:46.208Z)

```
Let me read the most relevant existing docs to ground the answer, then research data-efficiency
claims.
```

5. user (2026-06-08T13:04:47.160Z)

```
1       # Rally-detection methods ? research (2026-06-06)
2
3       Deep-research survey (23 sources, 110 claims ? 25 adversarially verified, 6 refuted, 12
final findings)
4       on alternative ways to find rally start/end in racket-sport video, for our **local-
first, permissive,
5       single-GPU** pipeline. Question: is Gemini-teacher ? local-distillation the best, or are
there competitive/
6       complementary methods?
7
8       ## Verdict
9       Teacher?student distillation is **sound but NOT uniquely best**. The evidence points to
a **complementary
10      multi-cue LOCAL stack**. VLM/MLLM teachers are validated specifically as **training-time
```

priors**, not runtime
localizers ? which is exactly our offline-Gemini framing. The single highest-leverage near-term lever is to
add a fully-local AUDIO hit-detection cue and fuse it with the shuttle trajectory; pose/court serve-gating
is the strong #2.

Three fully-local, permissive, competitive method families
1. **Audio hit detection (shuttle "thwack") ? top cheap lever.** Energy-peak + small-CNN gives sub-ms impact
timing (peak precision ~0.98, recall ~0.95+); classical MFCC+GMM is CPU-light. ? Only as good as its
post-processing: raw peaks ~41?50 F-score ? **likelihood-ratio confidence raises audio-only to ~68?77**;
fully-automatic on-court systems degrade (~85%, and one tennis system hit 46% precision). Mostly validated
on tennis/table-tennis (louder balls) ? shuttle SNR transfer is an open risk.
2. **Pose + court-geometry state machine** (= our "gate B" idea, validated). Serve-ready stance (court+pose
features, BiLSTM/CNN over a 12×26 hand-engineered matrix) ? rally **START**; shuttle-flight gradient
(?80% consecutive no-motion frames) ? rally **END**. Cheap (engineered features, no big model).
3. **Compact HitNet GRU** (MonoTrack, ~16K params, 2-layer GRU over 12-frame window of court corners + both
players' poses + shuttle xy) ? consumes exactly our available inputs; trivially runnable. Detects *strokes*
(a segmentation primitive), not whole rallies.

Key supporting evidence
- **Audio+visual fusion beats either alone:** tennis HMM rally **recall 83% (fused) vs 54% visual / 63% audio**
at ~unchanged precision; soccer late-fusion +4?7% mAP. This is the core case for adding audio.
- **Distillation is legit:** MLLM4WTAL uses the MLLM as a training-time prior only; SSL+KD (COMEDIAN) is
top-tier action-spotting. Our teacher?student is a valid backbone.
- **Our windowing is "coarse action spotting" (5?60 s tolerance), not frame-exact PES** ? don't over-engineer
for frame precision.

? Caveats (honest)
- **Domain transfer:** nearly all high numbers come from **BWF/tennis BROADCAST** footage (replays, score
graphics, director cuts) or curated lab audio. Our target is single-fixed-camera amateur video ? broadcast
CNNs (97?99%) and scoreboard-OCR are inapplicable/degrade.
- **Metric mismatch:** headline numbers are **per-frame / per-component accuracy, NOT IoU-0.5 boundary F1**.
The "~95% beats your 0.44" comparison was **refuted (0-3)** ? not comparable.
- **License gap (hard product req):** ShuttleSet, MonoTrack/HitNet weights, and WASB *weights* commercial
terms were NOT confirmed ? must verify before commercial use. (WASB code = MIT; weights = academic-trained.)
- Scoreboard/score-overlay OCR: usable only when a stable on-screen score exists ? mostly N/A for amateur clips.

Datasets
- **ShuttleSet** ? 104 sets / 3,685 rallies / 36,492 strokes / 44 BWF matches (2018?21), 18 shot classes +
hitting/player locations. Stroke-level structure for train/eval, but broadcast-domain + license TBD.

Ranked recommendation (local-first, permissive, single-GPU)
Gemini-teacher ? local-distillation is a valid **backbone to AUGMENT, not rely on

alone**. Cheapest local adds,
52   in order: **(1) audio hit detection fused with trajectory** (attacks the WASB recall
bottleneck with inputs we
53   already have); **(2) pose/court serve-ready START + flight-gradient END gating**; **(3)
a compact HitNet-style
54   GRU** on pose+trajectory+court. All MIT/permissive-compatible and CPU/GPU-light. Keep
Gemini offline-only.
55
56   ## Concrete cheap next experiments (with what we already have)
57   - **E1 ? audio recall probe:** extract audio ? energy-peak detection (high-pass + EMA
threshold) ? candidate
58   impact timestamps ? overlay hit density on existing WASB windows ? does audio recover
rallies WASB misses?
59   (zero new training; directly tests the recall bottleneck.)
60   - **E2 ? audio FP suppression:** add likelihood-ratio confidence (raw peaks ~41?50 F ?
~68?77 F).
61   - **E3 ? fuse into the distillation student:** add audio-hit-rate / time-since-last-hit
/ applause-energy as
62   features to the logistic-regression student ? re-eval at IoU 0.5.
63   - **E4 ? pose-free serve gate:** use the existing person detector for a coarse "both
players settled in court"
64   gate at window boundaries (proxy for the pose state machine, no pose model needed).
65
66   ## Open questions
67   1. ShuttleSet / MonoTrack-HitNet / WASB-weights **commercial license** terms (hard
requirement).
68   2. Does shuttle-"thwack" audio detection hold on **real amateur single-camera**
badminton (weaker SNR, gym
69   reverb), or degrade below the tennis ~73?77 F audio-only ceiling?
70   3. On OUR footage at IoU 0.5, do audio/pose cues actually fix the **WASB recall**
bottleneck ? or does WASB
71   miss the underlying trajectory regardless of how windows are derived?
72   4. Most label-efficient path given n=2: more Gemini labels vs audio/pose weak-
supervision vs a small
73   hand-labeled golden set ? biggest IoU-0.5 lift per labeling hour?
74
75   ## Sources (primary)
76   Springer 978-3-031-69073-0_15 (+ SN CompSci 6(5):447) · MonoTrack arXiv:2204.01899 ·
arXiv:2409.11760 (audio
77   peaks) · Kijak tennis AV-HMM (irisa) · ShuttleSet arXiv:2306.04948 · broadcast-CNN
arXiv:2308.12199 /
78   2307.16000 · MLLM4WTAL arXiv:2411.08466 · action-spotting survey arXiv:2505.03991 ·
WASB-SBDT (github nttcom)
79   · CoachAI license (github wywyWang).
80

6. user (2026-06-08T13:04:47.643Z)

1   # Golden-Set Generation & Vendor Process (v1)
2
3   **Status:** Design v1 · **Date:** 2026-05-30 · **Vendor shortlist added 2026-05-30
(§6)**
4   **Owner decisions captured:** licensed/owned video only · rally `[start,end)` depth ·
tiered turnaround (fast automated + slow human) · vendor bake-off · **Bangalore/India-local
vendor preference**
5
6   This document defines how we produce **golden-set ground truth** (hand-verified rally
boundaries) for the badminton highlights product, how we **select annotation vendors**, and how
that same machinery powers **on-demand / surprise regression testing** of the production tool.
7
8   > **Companion docs:** the scoring mechanics live in EVALUATION.md; the
annotated-corpus rationale is [ADR-002](DECISIONS.md); the decision record for *this* process is
ADR-006. Video-capture quality requirements (a dependency of trustworthy golden sets) are
tracked separately in VIDEO_RECORDING_GUIDELINES.md.
9

> **Update (2026-06-02):** §6 shortlist done; the founder selected **Roora** (Bengaluru) for the **initial paid pilot** ? a **single-vendor pilot first**, not the full bake-off below. Roora's site/capability/location are verified; **quality is TBD via the pilot**. **Step 0 (the annotation guideline) is authored** ? collector repo `docs/annotation/` (`ROORA_BRIEF.md` rally CSV schema + `ANNOTATION_GUIDE.md` illustrated guide + intake-manifest & NDA templates). Pilot scope is **3 sports** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is built. Still pending: the internal **gold key** and the vendor-dependent **GS-3 `HumanVendorProvider`** (GS-1/GS-2/GS-4 already landed ? see GOLDEN_SET_IMPLEMENTATION.md).

---

1. The keystone insight: one scorer, three jobs

A vendor's golden-set output is a list of rally `[start,end)` intervals ? **structurally identical to a prediction** from our pipeline. So the *exact same* deterministic scorer in `backend/eval/metrics.py` (`interval_iou` ? `match_intervals` ? `score`) grades all three of:

1. **Product vs. ground truth** ? the precision/recall/F1/IoU we already report (`EVALUATION.md`).
2. **Vendor vs. our internal answer key** ? the quality gate during vendor selection.
3. **Annotator vs. annotator** ? inter-annotator agreement (IAA), our proxy for "is this even a well-defined task?"

This symmetry is what makes the whole system small, auditable, and extensible. **No new scoring code is needed** ? only a new *source* of intervals (the vendor).

---

2. Why this also solves the privacy nuance

The monetization goal (hosted API) carries a "someone views the user's video" concern. This process sidesteps it **by construction**:

- **Golden sets are built only from footage we license or own** ? starting with the founder's **constantly-growing personal collection** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. **No end-user uploads ever flow to a vendor.**
- Regression tests therefore run against a **representative held-out corpus**, not literal user content.
- All video egress to any third party passes through a **single provider boundary** (§4), so the security controls (DPA, no-reuse, delete-on-delivery, access logging) live in exactly one place.

> **Honest tradeoff:** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore *strong evidence*, not a *guarantee*, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see VIDEO_RECORDING_GUIDELINES.md, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

---

3. Selecting vendors ? the bake-off

Step 0 ? Write the annotation guideline (highest-leverage artifact)

Before contacting anyone, write a precise, example-rich definition of **what a rally is**:

- **Start trigger:** e.g. first serve motion vs. shuttle leaving the racket ? pick one, show clips.
- **End trigger:** shuttle hits floor/net/out, or umpire call.
- **Edge cases:** lets/replays, service faults, between-point walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.

Rally boundaries are genuinely subjective; **ambiguity here is the #1 cause of bad golden sets.** This guideline belongs conceptually with the per-sport `SportProfile`, so it generalizes to tennis/cricket later.

> **Done (2026-06-02):** authored as the Roora **brief + illustrated guide** in the collector repo `docs/annotation/`. Defines the rally CSV schema and the three golden rules ? ignore all dead time; include **every** rally (incl. service faults & lets); a shot = one racket/paddle/bat contact ? plus an `ending_reason` taxonomy (winner / forced_error / unforced_error / service_fault / let / other, shared across net-separated sports) and per-sport detail incl. **singles & doubles** for badminton & pickleball.

Step 1 ? Build a small internal gold key

One trusted expert (initially: the founder) hand-annotates **5?10 licensed clips** to near-perfect. This is the answer key vendors are graded against ? kept private during the bake-off.

Step 2 ? Run the bake-off (parallel pilot)

Send the **same** pilot clips to **2?3 shortlisted vendors** (see §6, favoring Bangalore/India), withholding the key. Grade every submission with **our own harness**:

Dimension	How we measure it	Tool
Accuracy	vendor CSV vs. gold key ? P/R/F1, mIoU, boundary error	`eval.score()`
Consistency	2 annotators per vendor ? pairwise IoU (IAA)	`interval_iou`
Cost	quoted ? or $ per video-minute	vendor quote
Turnaround	actual vs. promised, per tier	observed
Ops friction	file-drop vs. API; compliance with our CSV format	`annotations.load_annotations()`
Security	DPA signed; DPDP/ISO posture; delete-on-delivery	contract review

Step 3 ? Decide on measured quality-per-dollar

The bake-off numbers *are* the decision ? not sales decks. Low IAA at a vendor means either a weak vendor **or** a vague guideline (Step 0); investigate before blaming the vendor.

Step 4 ? Onboard the winner(s)

Sign DPA + the security controls in §4; codify the guideline as their spec; agree SLAs for both tiers (§5).

---

4. The extensible architecture ? an `AnnotationProvider` registry

This codebase already uses pluggable registries (validators, segmenters, AI providers, sports). The idiomatic move is the same shape for annotation sourcing:

```python
class AnnotationProvider(ABC):
    def submit(self, video_ref, guideline, sport) -> str:        # returns job_id
        ...
    def poll(self, job_id) -> str:                               # queued|running|done|failed
        ...
    def fetch(self, job_id) -> List[Interval]:                   # canonical [(start,end), ...]
        ...
```

`fetch()` returns the **same `List[Interval]`** that `annotations.load_annotations()` already produces, so everything downstream is unchanged.

Three implementations cover today plus both tiers:

Provider	Role	Tier
`FileProvider`	Reads a pre-made CSV (formalizes today's manual / ShuttleSet flow)	?
`HumanVendorProvider`	The bake-off winner; authoritative ground truth, corpus

growth, deep audits | **Slow (days)** |
95 | `AutomatedProvider` | An independent strong model labels rallies programmatically |
Fast (hours) |
96
97   The tiers are just two providers behind one interface, selectable per call.
98
99   ---
100
101   ## 5. The regression automation (the explicit ask)
102
103   > *"given a video, get a golden set created and verify its precision and recall against
the production tool"*
104
105   This becomes a thin wrapper over machinery that **already exists**:
106
107   ```python
108   def regress(video, tier="fast", tolerance=0.05):
109   gt   = annotation_registry.get(tier).fetch(# NEW: the provider
110   annotation_registry.get(tier).submit(video, GUIDELINE, sport))
111   predictions = process(video)   # existing production
pipeline ? DB
112   s   = eval.score(predictions, gt)   # existing metrics.py,
unchanged
113   regressed   = (s.recall   < baseline[video].recall   - tolerance or
114   s.precision < baseline[video].precision - tolerance)
115   if regressed: alert(video, s, baseline[video])
116   return s
117   ```
118
119   Only **one new component** (the provider). `run_eval.py`, `harness.py`, and `metrics.py`
are reused verbatim. Snapshot a `baseline.json` per corpus video so "regression" is a measured
delta, not a vibe.
120
121   **Triggers** (all feed the same `regress()`): CI on pipeline changes · on-demand (the
"surprise" case) · scheduled cron · pre-release gate.
122
123   ### ?? The oracle problem (a real correctness trap)
124   The fast-tier `AutomatedProvider` **must be a different model lineage than production**.
If production becomes the offline ActionFormer segmenter (ADR-004) and the oracle is *also*
ActionFormer, they share blind spots ? the test shows green while both are wrong. Pairings that
stay honest:
125
126   - production = offline learned segmenter ? fast-tier oracle = **paid Gemini full-video**
(different lineage).
127   - production = Gemini hybrid ? fast-tier oracle = a **different VLM** or the slow human
tier.
128
129   Treat the fast tier as a **smoke alarm**, not ground truth: any fast-tier "regression"
escalates to the slow human tier for confirmation before it blocks a release.
130
131   ---
132
133   ## 6. Vendor shortlist (Bangalore/India-favoring)
134
135   > **Researched 2026-05-30 via a lean manual pass** (targeted web search + direct
site/source verification ? NOT the heavy automated workflow, which failed). **Confidence is
labelled per row.** **No pricing is listed because none could be verified** ? every vendor in
this space quotes per-project, so ?/clip is an offline-conversation item (see §6.3). Treat
single-source rows as *leads to verify on the call*, not vetted recommendations.
136
137   ### 6.1 Tier-1 candidates ? verified India presence + video capability (talk to these
first)
138
139 | Vendor | Location (verified) | Video / temporal labeling | Notes for the call |
Confidence |

|---|---|---|---|---|
iMerit	**Bengaluru** office (A101 & A118, 43 Residency Rd, 560025; tech hub opened Oct 2022); also Kolkata HQ + US	Yes ? image/video/text/audio; explicit Sports & Biomechanics domain on their site	Largest & most enterprise-ready (5,500+ annotators); has a real **Bangalore** footprint for in-person coordination; ask for a sports/temporal-segmentation reference + a paid pilot. Higher-touch ? likely higher cost.	**High**
TELUS International AI (ex-**Playment**)	Playment was **Bangalore-founded** (2015); acquired by TELUS 2021, team retained	Yes ? built on Playment's 2D/3D image+video+LiDAR CV pipeline	Strong CV/video pedigree, Bangalore roots; but now a large global BPO ? confirm they'll take a **small startup pilot** and who the India delivery contact is.	**High** (origin) / **Med** (current India ops shape)
Taskmonk	**Bengaluru**	Yes ? "Video" listed as a labeling modality; platform **+** managed-service workforce	Mid-size, India-local, does both tooling and service ? potentially the best **cost/agility** fit for a startup pilot. Verify temporal/event labeling specifically (their heritage is e-commerce catalog tagging).	**Med-High**

6.2 Tier-2 ? India video-annotation services, location/specialts to confirm on contact

Vendor	Stated location	Note	Confidence
Infolks	India (Kerala-based per third parties)	Targets startups/SMBs; image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate	**Med**
Anolytics	India + USA (since 2018)	General data-annotation incl. video; widely listed	**Med**
Objectways	HQ Scottsdale AZ; **India delivery in Coimbatore** (not Bangalore)	Does video/LiDAR; India delivery but not Bangalore-local	**Med**
Roora (Roora ML Pvt Ltd)	**Bengaluru** ? HSR Layout (verified, `rooragrp.com`)	Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot (2026-06-02)** ? existence/capability/location verified; **quality TBD via the pilot**.	**Verified** (capability) · pilot pending
Tika Data / **Zuru**	Claimed **Bangalore** (single-source listicle each)	Smaller/startup annotation shops ? unverified beyond one directory mention; verify they exist & do temporal video before investing time	**Low**

6.3 Explicitly NOT verified / out of scope

- **Pricing for everyone** ? all quote-only; do not assume. Get per-video-minute (or per-rally) quotes during the offline conversations.
- **Karya (Bengaluru)** ? confirmed local and reputable, but **speech/text/translation only, no video** ? **excluded** for our use case.
- **Sports/temporal track record** ? beyond iMerit's stated "Sports & Biomechanics" domain, vendors' specific *rally-boundary / temporal-event* experience is unconfirmed marketing copy; probe with the pilot (§3).
- **DPDP / DPA / ISO posture** ? not verified per-vendor; make it a checklist item in the bake-off (§3 Step 2 security row).

6.4 Recommendation

For the **offline conversations**, start with **iMerit** (Bangalore-local, enterprise QA, sports domain) and **Taskmonk** (Bangalore-local, likely more startup-friendly cost/agility) as the two primary contacts, with **TELUS/Playment** as a third if you want a heavyweight CV pedigree. Run the §3 bake-off across whichever 2?3 agree to a paid pilot, and decide on measured quality-per-rupee. **Verify every "Confidence: Med/Low" claim on the call before committing.**

> **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local, startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot doesn't clear the §3/§4 quality bar.

> Sources (manual pass, 2026-05-30): iMerit Bengaluru hub ? imerit.net press / PRNewswire (Oct 2022) + contact listings; Playment/TELUS ? BusinessWire & Inc42 (2021 acquisition, Bangalore-founded); Taskmonk & Karya locations ? company sites + AnalyticsIndiaMag / IndiaAI; others ? AnalyticsIndiaMag / Ziloservices / vendor sites. Pricing intentionally omitted (unverifiable).

```
169
170     ## 7. Suggested phasing
171
172     | Phase | Deliverable | Depends on |
173     |---|---|---|
174     | 0 | Annotation guideline + internal gold key (5?10 clips) | nothing ? can start now |
175     | 1 | Vendor bake-off ? pick winner(s) | §6 research + guideline |
176     | 2 | `AnnotationProvider` registry + `FileProvider` + `HumanVendorProvider` | winner
onboarded |
177     | 3 | `AutomatedProvider` (independent oracle) + `regress()` + baseline snapshots |
Phase 2 |
178     | 4 | Wire surprise triggers (CI / on-demand / scheduled) | Phase 3 |
179
180     Phase 0 is unblocked today and is the highest-leverage step regardless of vendor choice.
181
182     > **Status (2026-06-02):** Phase 0 guideline ? authored (collector `docs/annotation/`);
internal gold key still pending. Phase 1 is running as a **single-vendor Roora pilot** across
**3 sports** (badminton, pickleball, table tennis) rather than a multi-vendor bake-off. Phase
2's `FileProvider` (GS-1) and Phase 3's `regress()` + automated oracle (GS-2/GS-4) have already
landed; the main remaining slice is the **vendor-dependent `HumanVendorProvider`** (GS-3), with
trigger-wiring (GS-5) after it.
183
184     ---
185
186     ## 8. Open questions
187     1. **Guideline authorship** ? who owns the canonical rally definition, and is one
expert's key enough, or do we want 2-expert consensus for the answer key itself?
188     2. **Corpus representativeness** ? how well does the owned/licensed collection mirror
eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)
189     3. **Baseline drift** ? when production legitimately improves, how/when do we re-
snapshot `baseline.json` so old baselines don't raise false regressions?
190     4. **Fast-tier oracle cost** ? per-video Gemini cost at regression frequency (see
`archives/MONETIZATION_AUDIT.md` §4 for the per-video estimate).
191
```

7. assistant (2026-06-08T13:04:50.246Z)

Now let me look at the data contracts and the evaluation/metrics to understand the exact schema we have, plus check the audio findings doc.

8. user (2026-06-08T13:04:51.464Z)

```
1       # Annotation & Evaluation
2
3       How to measure how well the pipeline detects rallies, using hand-annotated
4       ground-truth timestamps. The harness scores predictions **already stored in the
5       SQLite DB** ? it makes no API calls and is fully deterministic, so you can run it
6       as often as you like for free.
7
8       ---
9
10      ## 1. What you annotate (and what it does / doesn't do)
11
12      You annotate, per video, the `[start, end)` time range of each **rally**. That's
13      all the evaluation harness needs.
14
15      > **Important:** these rally timestamps measure and tune **rally segmentation**
16      > ? i.e. *where* the pipeline says rallies begin and end. They do **not** train
17      > the shuttle detector (TrackNet/WASB), which learns from per-frame shuttle
18      > `(x, y)` coordinate labels, a different and much heavier annotation. Use rally
19      > timestamps as your **golden evaluation set**; only invest in frame-level
20      > coordinate labels if evaluation shows *detection* (not segmentation) is the
21      > bottleneck. See `docs/TRACKNET_WSL_SETUP.md`.
22
23      ## 2. Annotation file format (CSV)
```

````
One CSV per video, one rally per row:

```csv
start,end
00:01:12,00:01:39
102.5,131.0
```

- The `start,end` header is optional (auto-detected).
- Timestamps may be **decimal seconds** (`102.5`) or **colon-separated**
  `MM:SS` / `HH:MM:SS` (`00:01:12`). Forms can be mixed.
- Blank lines and `#` comments are ignored.
- Malformed rows (bad timestamp, `start >= end`) fail loudly rather than being
  silently skipped ? so labeling mistakes surface immediately.

Generate an empty template to fill in:

```bash
python run_eval.py --scaffold --annotations rallies.csv
```

## 3. Running the evaluation

First make sure the video has been processed so its detections are in the DB
(the pipeline keys each video by its file MD5). Then:

```bash
# Identify the video by file (its MD5 is computed for you)...
python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv

# ...or by the stored video_id (MD5) directly:
python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
```

Useful flags:

| Flag | Effect |
|------|--------|
| `--stage {candidates,final,both}` | Which prediction stage(s) to score (default `both`). |
| `--detector <name>` | Only score candidates from one detector (e.g. `yolo_hybrid`, `tracknet_hybrid`). |
| `--iou <float>` | Match threshold (default `0.5`). |
| `--pr-curve` | Also report an IoU sweep (0.1?0.9). |
| `--show-failures` | List missed (FN) and spurious (FP) rallies with timestamps. |
| `--json <path>` | Write a machine-readable report. |
| `--db <path>` | Override the DB path (default: `output/<output.db_name>` from `config.json`). |

## 4. Reading the output

```
================================================================
 Rally evaluation ? video 'demo'
 Ground-truth rallies: 3 | IoU threshold: 0.5
================================================================
 Stage         Pred   TP   FP   FN   Prec    Rec     F1   mIoU  dStart    dEnd
----------------------------------------------------------------
 candidates       3    3    0    0   1.00   1.00   1.00   1.00    0.0s    0.0s
 final            2    2    0    1   1.00   0.67   0.80   0.95    0.2s    0.5s
================================================================
 [final] missed rallies (false negatives): 1
   - 00:01:20?00:01:32
```
````

- **TP / FP / FN** ? true positives (matched), false positives (spurious predictions), false negatives (missed rallies).
- **Prec / Rec / F1** ? precision, recall, and their harmonic mean.
- **mIoU** ? mean temporal IoU over matched pairs (how tightly matched rallies overlap their ground truth).
- **dStart / dEnd** ? mean absolute boundary error in seconds (are rallies detected but trimmed in the wrong place?).

The key diagnostic: candidates vs. final

Scoring both stages tells you *where* to spend effort:

- **Candidates miss rallies the GT has** ? the **detector** is the bottleneck. Improve the shuttle model (e.g. fine-tune TrackNet/WASB with frame-level labels) or relax detection thresholds.
- **Candidates catch them but `final` is wrong/trimmed** ? the **segmentation / AI handoff** is the bottleneck. Tune thresholds (`velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration`) or the AI prompt ? no new labels needed.

In the example above, detection is perfect (candidates: recall 1.0) but the AI handoff dropped one rally (final: recall 0.67) ? so the fix is in segmentation, not detection.

5. How matching works

Predictions and ground truth are sets of `[start, end)` intervals. A prediction **matches** a ground-truth rally when their **temporal IoU** (overlap ÷ union) is at least the threshold. Matching is **greedy and one-to-one** ? highest-IoU pairs are claimed first, and each prediction/GT is used at most once, so a single detection can't be credited for two rallies. Unmatched predictions are false positives; unmatched ground-truth rallies are false negatives.

6. Code map

File	Role
`backend/eval/metrics.py`	Pure interval math: IoU, matching, P/R/F1, boundary error.
`backend/eval/annotations.py`	Load/validate the ground-truth CSV.
`backend/eval/harness.py`	Read predictions from the DB, score, format the report.
`run_eval.py`	CLI entry point.
`tests/test_eval_*.py`	Deterministic tests for all of the above.

7. Troubleshooting

Symptom	Cause & fix
`ERROR: database not found at output/...`	The harness only reads stored predictions ? it does **not** run the pipeline. **Process the video first** (e.g. `run_process_and_stitch.py`) so detections land in the DB, then re-run.
Every rally shows as a miss (FN), `Pred` is 0	This video hasn't been processed, or you used the wrong `video_id`. The pipeline keys videos by **file MD5** ? pass the *same* file via `--video`, or the exact stored `--video-id`.
`video_id` doesn't match although it's "the same" video	Re-encoding/trimming changes the bytes, hence the MD5. Use the exact file that was processed, or look up the stored id (it's printed during processing).
`candidates` stage is empty but `final` has data (or vice-versa)	Depends on which segmenter ran. `gemini` writes only `final` (no candidates); `yolo_hybrid`/`tracknet_hybrid` write both. Use `--stage final` for the gemini segmenter, and `--detector <name>` to scope candidates.
Recall looks low only at high `--iou`	Rallies *are* found but boundaries are loose ? check `dStart`/`dEnd`. Tune `stitching` padding or the segmentation thresholds rather than the detector.

```
139     | `ValueError: start (...) >= end (...)` while loading annotations | A bad row in your
CSV. Timestamps must be increasing per rally and use seconds or `HH:MM:SS`. |
140
141     Run `python run_eval.py -h` for a quick reminder of this prerequisite and all flags.
142
```

9. user (2026-06-08T13:04:51.640Z)

```
<system-reminder>[Truncated: PARTIAL view ? showing lines 1-263 of 327 total (26410 tokens, cap
25000). Call Read with offset=264 limit=263 for the next page, or Grep to find a specific
section. Do NOT answer from this page alone if the answer may be further in the file.]</system-
reminder>

1       # CODE_MAP.md
2
3       **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5       **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7       **LEGEND**
8       - `@path` = file (always repo-relative)
9       - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10      - `->` = dataflow / call / "produces"
11      - `§` = data contract (canonical shape)
12      - `?` = gotcha / footgun
13      - `?` = registry / plugin extension point
14      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16      ---
17
18      ## 1. PIPELINE
19
20      ### 1A. Runtime: video file -> highlight reel
21      ```
22      typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
23                                              @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
24        -> video_id = MD5(whole file)         @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
25        -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
26              FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
27              [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
28        -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29        -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
30        -> seg = segmenter_registry.get(seg_name)(db, global_config)   ?
@backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
31        -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32              [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33        -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
34        -> select segment ids -> [(start,end)] (+ stitching padding)
35        -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
```

```
@backend/pipeline/stitcher/compiler.py
            per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
      ```
      Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
      ```
      P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
       -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted
via db.add_candidate_segment]
       -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt)   ? provider_registry
       -> P4 db.add_play_segment + db.link_candidate_to_segment
      ```
      Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
      Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).

      ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
      ```
      WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
       -> WasbRunner.healthcheck()  (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
       -> WasbRunner.run_predict(video, out_dir)
            stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
            -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
       -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py  (shared, re-exported by WasbRunner)
       -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING
BRAIN]
      ```
      (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)

      ### 1C. Calibration / eval flow (NO pipeline run unless told)
      ```
      GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
      DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage
? {candidates, final}
        -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
        -> harness.evaluate -> EvalReport -> report_to_dict
             -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
             -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py
[CI gate]
      Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate}  @backend/eval/calibration.py
      Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier)  @backend/eval/{calibrate_local,distill_local}.py
      Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
      Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies}  @backend/eval/gemini_refine.py
      Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
      ```
      Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
```
```

75
76   ---
77
78   ## 2. SUBSYSTEMS
79
80   ### 2.0 Orchestration & config
81   - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
82   - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
83   - `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read).
84   - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
85   - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86   - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
87   - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
88   - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89   - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
90   - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.
91
92   - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config import load_config, AppConfig`.
93   - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
94   - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95   - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
96   - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,

`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
97 - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal of 5 sports. `::Config = AppConfig` alias.
98 - `@backend/config/loader.py` ? load/save with defaults-merge.
99 - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
100 - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
101 - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
102 - `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
103 - `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105 ### 2.1 Segmenters (under pipeline/segmenters) ?
106 - `@backend/pipeline/segmenters/base.py`
107 - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
108 - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
109 - `@backend/pipeline/segmenters/__init__.py` ? imports all 5 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
110 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config `indexing.wasb.velocity_threshold(0.02)` + shared `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs `{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit (copy-paste drift).
111 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
112 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model` (lazy `torchvision` import so module/tests load without it),

`::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU); `yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated aliases still honored.
113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
115
116   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117   - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
118   - `@backend/pipeline/detectors/tracknet_runner.py`
119   - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120   - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
121   - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
122   - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC for future swappability.
123   - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
124   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local / distill_local / export_wasb_segments.
125
126   ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127   - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
128   - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`

CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129   - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. §
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>~0.1` = overfit alarm.
130   - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131   - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. §
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132   - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133   - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
134   - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135   - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
136   - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
137   - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
138   - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
139   - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
`backend.annotations` NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
140

2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)

- `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength), `::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`; `k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted (PR #35).
- `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion). `::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally), `::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses `hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.

2.4 Stitcher / tools / utils

- `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw, segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time` (mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so final concat can stream-copy ? change a preset and concat may break.
- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows), `::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
- `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure = "unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires `reencode=True`.
- `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s; ? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.

2.5 Validators ?

- `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ? registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path, config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a validator**: subclass + `registry.register(MyValidator())`.
- `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator, ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator, RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global registry as side effect).
- `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring `'mp4'` (so MOV-family passes); external `ffprobe`.
- `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator` (`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if absent). `min_fps` 29.9; min res 1280x720.
- `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator` (`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded (comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES (too short).
- `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail threshold.
- `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var < `min_blur_threshold`(default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ? magic absolute, not resolution-normalized.
- `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`; 3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport -> empty cfg, falls back to defaults, does NOT fail.

160
161   ### 2.6 Sports / Providers / Annotations ?
162   - `@backend/sports/base.py::SportProfile` ABC
(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `.name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. § prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
163   - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `'gemini'`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. § `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
164   - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch ->
List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable.
165   - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the
resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; ->
`backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
166   - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub;
injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only
warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via
`annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only
because no-arg ctor succeeds.
167   - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of
file_provider/automated_provider are load-bearing (removing de-registers providers).
168
169   ### 2.7 Infrastructure (DB) ?
170   - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
(`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
`::_init_db` (currently =1, `version<1` creates `candidate_segments`). `::_get_connection` (?
re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped).
Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_s
egment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`,
`clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point
(common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
`idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES
dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments`
filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow
exceptions.
171
172   ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
173   - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path,
config, val_results, val_context=None, segmenter_requested=, segmenter_actual=,
was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed
config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if
unavailable/disabled; pulls play_segments/candidates from `db` (read-only
`query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to
`harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps
everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s
`finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video.
`::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import
harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default
True.
174   - `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the

footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`.
`::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now also `audio_present` for Input Quality reports extension; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
`::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so pure helpers like video_meta are importable for tests/coverage even without the dep.)
175 - `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules: `silent_provider_noop`(error: no api key + 0 segs), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).
176
177 ---
178
179 ## 3. DATA CONTRACTS (canonical shapes ? one place)
180
181 § **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
182 `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
183 -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
184
185 § **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo `_extract_action_windows_from_chunk`):
186 `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float, mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
187
188 § **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
189 `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)), stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}` (stats/detection_params deserialized on read).
190 `detection_params` keys by detector: wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_threshold,min_action_window_duration}`.
191
192 § **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
193 `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server, receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count, shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare `model_name`; motion adds random `intensity/avg_speed_kmh`.
194
195 § **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id` stamped internally).
196
197 § **GT / golden CSVs** (TWO conventions ? do not conflate):
198 - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.
199 - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,

`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
200
201   § **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
202   § **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
203   § **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},
...]}`; ? DB key = file MD5, manifest `video_id` is a label.
204
205   § **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
206
207   § **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
208
209   § **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
210
211   § **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
212
213   ---
214
215   ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
216   - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
217   - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
218   - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
219   - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
220   - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
221   - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
222
223   ---
224
225   ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
226   - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest

tracks the exact file used.
227   - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
228   - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
229
230   ---
231
232   ## 6. GOTCHAS (cross-cutting footguns)
233   - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
234   - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
235   - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'`
getattr fallback noted above.
236   - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
237   - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output.output_dir` ? honored end-to-end.
238   - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
returns `[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
239   - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions
(designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a
SOFT dep ? absent -> no-op.
240   - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
241   - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
242   - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
243   - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
modules).
244   - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
245   - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
246   - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
correlation are NOT config-overridable.
247   - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
248   - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
249   - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
250   - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared

API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`; `harness.get_predictions` reused by `regression.regress`.
251   - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
252   - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
253   - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
254   - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.
255
256   ---
257
258   ## 7. RAMP PATHS (to do X -> open Y::symbol)
259 | Task | Open |
260 |---|---|
261 | Change a config default / add a config field | `@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
262 | Read config in new code | `from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
263 | Tune candidate windowing (velocity/merge/min-dur) | `@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration` |

10. assistant (2026-06-08T13:04:55.560Z)

I have a strong grounding on our data shapes and pipeline. Now let me research the data-efficiency claims. I'll fan out web searches on the key topics: TAL data formats, few-shot/data-efficient TAL, semi-supervised TAL / pseudo-labeling, active learning for TAL, and VLM video-QA fine-tuning data requirements.

11. user (2026-06-08T13:05:02.865Z)

Web search results for query: "temporal action localization dataset annotation format ActionFormer THUMOS ActivityNet JSON segments"

Links: [{"title":"GitHub - BharathChalla/ActionFormer: ActionFormer (ECCV 2022) · GitHub","url":"https://github.com/BharathChalla/ActionFormer"},{"title":"GitHub - wangcheng666/ActionFormer: Temporal Action Localization Enhanced through Cross-Modal and Cross-Structural Distillation · GitHub","url":"https://github.com/wangcheng666/ActionFormer"},{"title":"GitHub - csh415/actionformer_performance","url":"https://github.com/csh415/actionformer_performance"},{"title":"actionformer_release/README.md at main · happyharrycn/actionformer_release","url":"https://github.com/happyharrycn/actionformer_release/blob/main/README.md"},{"title":"GitHub - yafeng19/actionformer: Code release for ActionFormer (ECCV 2022)","url":"https://github.com/yafeng19/actionformer"},{"title":"GitHub - chrisindris/actionformer_release: Code release for ActionFormer (ECCV 2022) · GitHub","url":"https://github.com/chrisindris/actionformer_release"},{"title":"GitHub - happyharrycn/actionformer_release: Code release for ActionFormer (ECCV 2022) · GitHub","url":"https://github.com/happyharrycn/actionformer_release"},{"title":"actionformer_release/README.md at main · minzhang15/actionformer_release","url":"https://github.com/minzhang15/actionformer_release/blob/main/README.md"},{"title":"PcmNet: Position-Sensitive Context Modeling Network for Temporal Action Localization","url":"https://arxiv.org/pdf/2103.05270"},{"title":"Pseudo label refining for semi-supervised temporal action localization","url":"https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11798530/"}]

Based on the search results, here's what I found about temporal action localization dataset annotation formats for ActionFormer, THUMOS, and ActivityNet:

Key Findings

ActionFormer uses action annotations in JSON format similar to the ActivityNet annotation format. This format is consistently used across datasets that ActionFormer supports.

Datasets Supported

ActionFormer is a Transformer-based model for temporal action localization that detects the onsets and offsets of action instances and recognizes their action categories, achieving 71.0% mAP at tIoU=0.5 on THUMOS14.

ActionFormer also demonstrates strong results on ActivityNet 1.3 (36.56% average mAP) and EPIC-Kitchens 100.

The search results confirm that the JSON annotation format is used across multiple datasets (THUMOS, ActivityNet, and EPIC-Kitchens), but the specific details about the JSON segment structure and schema are not explicitly provided in the retrieved results. For more detailed information about the exact JSON schema with segment definitions, you may want to consult the ActionFormer GitHub repository directly or the ActivityNet dataset documentation.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. user (2026-06-08T13:05:03.091Z)

Web search results for query: "semi-supervised temporal action localization pseudo-labeling few labeled videos data efficiency"

Links: [{"title":"Towards Adaptive Pseudo-label Learning for Semi-Supervised Temporal Action Localization","url":"https://arxiv.org/html/2407.07673v1"},{"title":"Pseudo label refining for semi-supervised temporal action localization","url":"https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11798530/"},{"title":"Towards Adaptive Pseudo-label Learning for Semi-Supervised Temporal Action Localization","url":"https://arxiv.org/pdf/2407.07673"},{"title":"Learning from Noisy Pseudo Labels for Semi-Supervised ...","url":"https://openaccess.thecvf.com/content/ICCV2023/papers/Xia_Learning_from_Noisy_Pseudo_Labels_for_Semi-Supervised_Temporal_Action_Localization_ICCV_2023_paper.pdf"},{"title":"Pseudo label refining for semi-supervised temporal action localization - PMC","url":"https://pmc.ncbi.nlm.nih.gov/articles/PMC11798530/"},{"title":"Pseudo label refining for semi-supervised temporal action localization | PLOS One","url":"https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0318418"},{"title":"Dual Guidance Semi-Supervised Action Detection","url":"https://arxiv.org/html/2507.21247"}]

I found relevant research on semi-supervised temporal action localization with pseudo-labeling techniques. Here are the key findings:

Overview

Semi-supervised TAL (SS-TAL) methods leverage a large number of unlabeled videos and only a few labeled videos for model training, addressing data efficiency challenges in the field.

Data Annotation Challenge

The training of temporal action localization models relies heavily on a large amount of manually annotated data, and video annotation is more tedious and time-consuming compared with image annotation. This motivates the use of semi-supervised approaches.

Pseudo-Labeling Methods

Pseudo-labeling methods exploit unlabeled data by training on self-generated predictions (pseudo labels), but they are susceptible to low-quality pseudo labels due to inaccuracies in the model's predictions.

Recent approaches address this issue. For example, Adaptive Label Quality Assessment (ALQA)

jointly learns classification confidence and localization reliability, followed by dynamically selecting pseudo labels based on the joint score.

Another method proposes a pseudo-label refining (PLR) framework based on a teacher-student approach with pseudo-label self-refinement to improve boundary accuracy, boundary synthesis to refine temporal intervals, and adaptive weight learning for progressively learning pseudo labels with different qualities.

SSL Paradigms

Consistency regularization and pseudo-labeling are two main paradigms in semi-supervised learning, with both being applied to temporal action localization tasks.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

13. assistant (2026-06-08T13:05:05.640Z)

Let me fetch the ActivityNet/ActionFormer JSON format precisely, and search for active learning + VLM video fine-tuning data needs.

14. user (2026-06-08T13:05:13.351Z)

ActionFormer JSON Annotation Format

Based on the README, the document states annotations follow "similar to ActivityNet annotation format" but does not provide the exact JSON structure, top-level keys, or specific segment field details.

What Is Specified

The README mentions:
- **Datasets supported**: THUMOS14, ActivityNet 1.3, EPIC-Kitchens 100, and Ego4D Moment Queries
- **Segment representation**: Features are extracted at specific frame rates and strides, producing "one feature vector" at regular temporal intervals (e.g., ~0.1333 seconds for THUMOS14)
- **Training data scale**: The document does not explicitly state how many labeled training videos THUMOS14 or ActivityNet 1.3 contain

Information Gaps

The README does not provide:
- Exact JSON schema with top-level keys
- Per-video field specifications (subset, duration, fps)
- Segment format (whether in seconds or frames)
- Per-annotation fields beyond action category

To obtain this information, you would need to consult the ActivityNet annotation documentation directly or examine the code in `./libs/datasets/` within the repository.

15. user (2026-06-08T13:05:16.655Z)

Web search results for query: "active learning temporal action localization which segments to annotate next reduce labeling cost"

Links: [{"title":"Active learning with effective scoring functions for semi-supervised temporal action localization - ScienceDirect","url":"https://www.sciencedirect.com/science/article/abs/pii/S0141938223000677"},{"title":"[2208.14856] Active Learning with Effective Scoring Functions for Semi-Supervised Temporal Action Localization","url":"https://arxiv.org/abs/2208.14856"},{"title":"Active learning method for temporal action localization in untrimmed videos","url":"https://image-

ppubs.uspto.gov/dirsearch-public/print/downloadPdf/10726313"},{"title":"What do I Annotate Next? An Empirical Study of Active ...","url":"https://joonyoung-cv.github.io/assets/paper/18_eccv_what_do.pdf"},{"title":"Two-Stage Active Learning for Efficient Temporal Action Segmentation Yuhao Su","url":"https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/06348.pdf"},{"title":"A flexible model for training action localization with varying levels of supervision","url":"https://arxiv.org/pdf/1806.11328"},{"title":"Boundary-Centric Active Learning for Temporal Action Segmentation","url":"https://arxiv.org/html/2604.15173v1"},{"title":"Weakly Supervised Temporal Action Localization with Segment-Level Labels | Springer Nature Link","url":"https://link.springer.com/chapter/10.1007/978-3-030-88004-0_4"},{"title":"What Do I Annotate Next? An Empirical Study of Active Learning for Action Localization | Springer Nature Link","url":"https://link.springer.com/chapter/10.1007/978-3-030-01252-6_13"},{"title":"Action-Agnostic Point-Level Supervision for Temporal Action Detection","url":"https://arxiv.org/html/2412.21205v1"}]

Based on the search results, here's what research shows about active learning for temporal action localization and reducing labeling costs:

Active Learning Approaches for Temporal Action Localization

A key practical challenge is the large amount of labeled training data required, which motivates semi-supervised temporal action localization with active learning methods to reduce expensive human label cost.

Core Strategy

Active learning techniques overcome the labeling bottleneck by intelligently selecting samples from unlabeled datasets to be annotated by an oracle, allowing an active learner to achieve high accuracy using as few labeled samples as possible and thereby minimizing the cost of obtaining labeled training data.

Selection Mechanisms

Research proposes several approaches for deciding which segments to annotate:

1. **Uncertainty-based Selection**: Scoring functions that consider the uncertainty of the localization model are used, such as Temporal Proposal Entropy (TPE) which takes entropy of predicted label distribution as a measure of uncertainty.

2. **Temporal Aggregation**: For dense prediction tasks, informativeness can be localized, motivating region-level acquisition and cost-aware annotation policies. In temporal action segmentation, effective acquisition requires per-timestep uncertainty estimation and temporal aggregation strategies that map dense uncertainty into clip/video selection under realistic annotation budgets.

3. **Sequence Diversity**: Some approaches prioritize unlabeled videos based on high alignment cost to labeled prototypes to encourage diversity in temporal structure, and select clips via alignment-based summarization objectives to promote sequential coverage within each queried video.

Performance Impact

Compared to passive learning with random sampling, active learning achieves better performance by prioritizing labeling informative samples when given limited label budget, and large label cost will be saved when achieving expected localization performance.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

16. assistant (2026-06-08T13:05:20.582Z)

Let me get the precise ActivityNet JSON schema from the ActionFormer data loader, and search for

VLM video fine-tuning data requirements and self-training/teacher-student data efficiency numbers.

17. user (2026-06-08T13:05:26.900Z)

THUMOS14 JSON Structure

Top-level Structure

The dataset expects a JSON file with a ``'database'`` key containing video entries.

Per-video Fields

Each video object contains:
- ``'subset'``: The dataset split (e.g., training, validation, test)
- ``'fps'``: Frames per second (optional if `default_fps` is provided)
- ``'duration'``: Video duration in seconds (optional, defaults to 1e8)
- ``'annotations'``: Array of action segments (optional)

Per-annotation Fields

Each annotation object within the ``'annotations'`` array contains:
- ``'segment'``: A two-element array representing `[start_time, end_time]` in seconds
- ``'label'``: The action category name (string)
- ``'label_id'``: Numeric identifier for the action class

The code processes these by converting temporal coordinates from seconds into "temporal feature grids" based on the feature stride and frames per second.

18. user (2026-06-08T13:05:38.201Z)

Web search results for query: "Qwen2-VL Qwen2.5-VL video temporal grounding fine-tuning number of training samples LoRA data requirements"

Links: [{"title":"How to Fine-Tune Qwen2.5-VL | Datature Blog","url":"https://datature.io/blog/how-to-fine-tune-qwen2-5-vl"},{"title":"Spatio-Temporal Token Pruning for Efficient High-Resolution GUI Agents","url":"https://arxiv.org/pdf/2602.23235"},{"title":"Qwen2.5-VL: Multimodal Vision-Language Model","url":"https://www.emergentmind.com/topics/qwen2-5-vl"},{"title":"Video Understanding and Grounding with Qwen 2.5 - PyImageSearch","url":"https://pyimagesearch.com/2025/06/16/video-understanding-and-grounding-with-qwen-2-5/"},{"title":"TEMPURA: Temporal Event Masked Prediction and Understanding for Reasoning in Action","url":"https://arxiv.org/pdf/2505.01583"},{"title":"TempSamp-R1: Effective Temporal Sampling with Reinforcement Fine-Tuning for Video LLMs","url":"https://arxiv.org/pdf/2509.18056"},{"title":"VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://arxiv.org/pdf/2504.06958"},{"title":"GitHub - QwenLM/Qwen3-VL: Qwen3-VL is the multimodal large language model series developed by Qwen team, Alibaba Cloud. · GitHub","url":"https://github.com/QwenLM/Qwen3-VL"},{"title":"VideoPerceiver: Enhancing Fine-Grained Temporal Perception in Video Multimodal Large Language Models","url":"https://arxiv.org/pdf/2511.18823"},{"title":"March 5, 2025 Qwen2.5-VL Technical Report Qwen Team, Alibaba Group","url":"https://arxiv.org/pdf/2502.13923"}]

Let me search for more specific information about LoRA data requirements and training sample numbers.

Links: [{"title":"(PDF) Fine-Tuning the Qwen2.5-VL Model for Intelligent Applications in the Electrical Domain","url":"https://www.researchgate.net/publication/395888307_Fine-Tuning_the_Qwen25-VL_Model_for_Intelligent_Applications_in_the_Electrical_Domain"},{"title":"VLM LoRA Training | Qwen2.5-VL | Raw text + Images + Video · Issue #2026 · unslothai/unsloth","url":"https://github.com/unslothai/unsloth/issues/2026"},{"title":"WaymoQA: A Multi-View Visual Question Answering Dataset for Safety-Critical Reasoning in Autonomous Driving","url":"https://arxiv.org/pdf/2511.20022"},{"title":"Fine-Tuning the Qwen2.5-7B-VL-Instruct Model: A Comprehensive Guide","url":"https://connectaman.hashnode.dev/fine-tuning-the-qwen25-7b-vl-instruct-model-a-comprehensive-guide"},{"title":"Qwen 2.5-VL 7B Fine-Tuning Guide

for Segmentation","url":"https://www.labellerr.com/blog/fine-tune-qwen-2-5-vl/"},{"title":"How to Fine-Tune Qwen2.5-VL with a Custom Dataset","url":"https://blog.roboflow.com/fine-tune-qwen-2-5/"},{"title":"GitHub - 2U1/Qwen-VL-Series-Finetune: An open-source implementaion for fine-tuning Qwen-VL series by Alibaba Cloud. · GitHub","url":"https://github.com/2U1/Qwen-VL-Series-Finetune"},{"title":"Animation Needs Attention: A Holistic Approach to Slides Animation Comprehension with Visual-Language Models","url":"https://arxiv.org/pdf/2507.03916"}]

Let me search for more specific information about minimum dataset sizes and training data requirements for temporal grounding tasks.

Links: [{"title":"Enrich and Detect: Video Temporal Grounding with Multimodal LLMs","url":"https://arxiv.org/pdf/2510.17023"},{"title":"arXiv Preprint Datasets and Recipes for Video Temporal Grounding via","url":"https://arxiv.org/pdf/2507.18100"},{"title":"Datasets and Recipes for Video Temporal Grounding via Reinforcement Learning","url":"https://arxiv.org/html/2507.18100v1"},{"title":"Grounded-VideoLLM: Sharpening Fine-grained Temporal Grounding in Video Large Language Models","url":"https://arxiv.org/html/2410.03290v2"},{"title":"TimeSuite: Improving MLLMs for Long Video Understanding via Grounded Tuning","url":"https://arxiv.org/pdf/2410.19702"},{"title":"TimeExpert: An Expert-Guided Video LLM for Video Temporal Grounding","url":"https://openaccess.thecvf.com/content/ICCV2025/papers/Yang_TimeExpert_An_Expert-Guided_Video_LLM_for_Video_Temporal_Grounding_ICCV_2025_paper.pdf"},{"title":"UniversalVTG: A Universal and Lightweight Foundation Model for Video Temporal Grounding","url":"https://arxiv.org/pdf/2604.08522"},{"title":"TempSamp-R1: Effective Temporal Sampling with Reinforcement Fine-Tuning for Video LLMs","url":"https://arxiv.org/pdf/2509.18056"},{"title":"Planner-Refiner: Dynamic Space-Time Refinement for Vision-Language Alignment in Videos","url":"https://arxiv.org/pdf/2508.07330"}]

Based on my web search results, I found relevant information about fine-tuning Qwen2-VL and Qwen2.5-VL for video temporal grounding tasks. Here's what I found:

Video Temporal Grounding Capabilities

Qwen2.5-VL has robust long video understanding with temporal resolution scaling that allows event-level grounding in videos spanning hours. The model introduces dynamic FPS sampling and upgrades MRoPE in the temporal domain by aligning to absolute time, thereby facilitating more sophisticated temporal sequence learning.

Training Data Requirements

For video temporal grounding tasks, research papers show varying dataset sizes:

- The TVG-RL-18K dataset is used for RL training and the TVG-Coldstart-13K dataset for SFT-based coldstart.
- A TVG-Coldstart-13K dataset is used for SFT-based coldstart.
- One constructed dataset comprises 17K samples for training.

LoRA Fine-tuning Configuration

For LoRA-based fine-tuning of Qwen2.5-VL:

LoRA is applied to target modules such as the self-attention components (q_proj, k_proj, v_proj) and the MLP components (up_proj, down_proj, gate_proj), with LoRA configurations including rank=8/16, lora_alpha=16, and lora_dropout=0.05.

The base weights of the language and vision towers and the projector are frozen, and LoRA adapters are applied to all supported modules except the LM head and token embeddings.

Training Hyperparameters

Training hyperparameters involve the AdamW optimizer with a learning rate of 5e-5 with cosine scheduling, a per-GPU batch size of 4 (at 4 FPS) or 8 (at 1 or 2 FPS), and 5 epochs of training.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

19. assistant (2026-06-08T13:05:42.353Z)

I now have the ActivityNet/ActionFormer JSON schema precisely, plus VTG fine-tuning data scales (13K-18K samples). Let me get a couple more data-efficiency anchors: the "Datasets and Recipes for VTG via RL" paper (RL data efficiency) and self-training/noisy-student data-efficiency numbers, plus point-level/timestamp supervision efficiency.

20. user (2026-06-08T13:05:50.180Z)

Data Efficiency Analysis: SFT Cold-Start vs RL in VTG-R1

Training Sample Allocation

The paper uses **distinct dataset sizes** for each stage:

- **SFT Cold-Start**: 13,000 samples (TVG-Coldstart-13K)
- **RL Stage**: 18,000 samples (TVG-RL-18K)

Data Efficiency Claims

The authors do **not explicitly claim that RL achieves better data efficiency** than SFT alone. Instead, they argue that **both stages are complementary and necessary**:

> "high-quality cold-start data and difficulty-controlled RL training are both crucial for improving model performance"

Key Finding on Data Quality Over Quantity

The paper demonstrates that **quality matters more than quantity**:

> "selecting high-quality cold start data is more effective for learning robust reasoning abilities than simply increasing the quantity of training data"

When comparing filtered (13K) vs. unfiltered (56K) cold-start data, the filtered version significantly outperformed despite using fewer samples.

Limitation: RL Computational Cost

The authors acknowledge a tradeoff: RL training introduces "considerable computational overhead," suggesting it's not necessarily more *data*-efficient, but rather achieves better performance when combined with quality initialization data.

21. user (2026-06-08T13:05:55.483Z)

Web search results for query: "timestamp supervision point-level annotation temporal action localization 6x faster than full segment annotation efficiency"

Links: [{"title":"Point-Level Temporal Action Localization: Bridging Fully-supervised Proposals to Weakly-supervised Losses | Request PDF","url":"https://www.researchgate.net/publication/347300936_Point-Level_Temporal_Action_Localization_Bridging_Fully-supervised_Proposals_to_Weakly-supervised_Losses"},{"title":"HR-Pro: Point-supervised Temporal Action Localization","url":"https://arxiv.org/pdf/2308.12608"},{"title":"Action-Agnostic Point-Level Supervision for Temporal Action Detection","url":"https://arxiv.org/html/2412.21205v1"},{"title":"Learning Reliable Dense Pseudo-Labels for Point-Level Weakly-Supervised Action Localization | Neural Processing Letters","url":"https://link.springer.com/article/10.1007/s11063-024-11598-w"},{"title":"[2012.08236] Point-Level Temporal Action Localization: Bridging Fully-supervised Proposals to Weakly-supervised Losses","url":"https://arxiv.org/abs/2012.08236"},{"title":"Point-Level Temporal

Action Localization: Bridging Fully-supervised Proposals to Weakly-supervised Losses","url":"https://arxiv.org/pdf/2012.08236"},{"title":"LPR: learning point-level temporal action localization through re-training | Multimedia Systems | Springer Nature Link","url":"https://link.springer.com/article/10.1007/s00530-023-01128-4"},{"title":"Point-Level Temporal Action Localization: Bridging Fully-supervised Proposals to Weakly-supervised Losses - ADS","url":"https://ui.adsabs.harvard.edu/abs/2020arXiv201208236J/abstract"},{"title":"Temporal Action Segmentation from Timestamp Supervision","url":"https://arxiv.org/pdf/2103.06669"},{"title":"CITAL: Counterfactual intervention for temporal action localization with point-level annotation - ScienceDirect","url":"https://www.sciencedirect.com/science/article/abs/pii/S0925231225006782"}]

I've performed the web search for your query about timestamp supervision, point-level annotation, temporal action localization, and efficiency improvements. Here are the key findings from the search results:

Key Concepts Found:

Point-Level temporal action localization (PTAL) aims to localize actions in untrimmed videos with only one timestamp annotation for each action instance.

Point-level annotations refer to the practice of annotating only a single timestamp and action class for each action instance during training. In comparison to traditional weakly supervised temporal action localization methods that solely rely on video-level labels, the additional cost is limited, yet the performance improvement is significant.

Efficiency and Annotation Advantages:

The search results demonstrate that point-level supervision offers significant advantages in annotation efficiency. To achieve better trade-off between the annotation costs and detection accuracy, researchers have proposed action-agnostic point-level (AAPL) supervision, a novel form of weak supervision for temporal action detection.

However, I should note that while the search results extensively discuss point-level annotation methods and their efficiency advantages over full segment annotation, **I did not find a specific reference to the exact "6x faster" metric mentioned in your search query**. The results confirm that point-level annotation is significantly more efficient than full segment annotation, but that specific quantitative comparison may be from a particular paper that didn't appear in these search results.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

22. assistant (2026-06-08T13:05:58.563Z)

Let me get the specific timestamp/point supervision annotation cost numbers (the well-known SF-Net / Ma et al. and the timestamp-supervision paper claim), and one more on noisy-student / self-training data efficiency and temporal data augmentation for TAL.

23. user (2026-06-08T13:06:09.478Z)

Web search results for query: "noisy student self-training teacher student action recognition unlabeled video data efficiency improvement"

Links: [{"title":"Source-free video domain adaptation by learning from noisy labels - ScienceDirect","url":"https://www.sciencedirect.com/science/article/abs/pii/S0031320324010793"},{"title":"Semi-Supervised Teacher-Reference-Student Architecture for Action Quality Assessment","url":"https://arxiv.org/html/2407.19675v1"},{"title":"Noisy Student (EfficientNet) · Hugging Face","url":"https://huggingface.co/docs/timm/models/noisy-student"},{"title":"Pseudo label refining for semi-supervised temporal action localization - PMC","url":"https://pmc.ncbi.nlm.nih.gov/articles/PMC11798530/"},{"title":"What is Noisy Student Training? | Activeloop Glossary","url":"https://www.activeloop.ai/resources/glossary/noisy-student-training/"},{"title":"Semi-Supervised Noisy Student Pre-training on EfficientNet

Architectures for Plant Pathology Classification","url":"https://arxiv.org/pdf/2012.00332"},{"title":"Cross-Domain HAR: Few Shot Transfer Learning for Human Activity Recognition","url":"https://arxiv.org/pdf/2310.14390"},{"title":"DTG-Net: Differentiated Teachers Guided Self-Supervised Video Action Recognition","url":"https://arxiv.org/pdf/2006.07609"},{"title":"Self-training with Noisy Student improves ImageNet classi?cation","url":"https://openaccess.thecvf.com/content_CVPR_2020/papers/Xie_Self-Training_With_Noisy_Student_Improves_ImageNet_Classification_CVPR_2020_paper.pdf"},{"title":"Pseudo Label Is Better Than Human Label","url":"https://arxiv.org/pdf/2203.12668"}]

Based on the search results, I found comprehensive information about noisy student self-training, teacher-student architectures, and their application to action recognition with unlabeled video data:

Key Concepts

Noisy Student Training is a semi-supervised learning approach that extends the idea of self-training and distillation with the use of equal-or-larger student models and noise added to the student during learning.

How It Works

A teacher model is trained using labeled images, then used to generate pseudo labels on unlabeled images. A student model minimizes combined cross entropy loss on both labeled and unlabeled images. Finally, the process is iterated by putting back the student as a teacher to generate new pseudo labels and train a new student.

Application to Action Recognition

A self-supervised teacher-student architecture can be used for video action recognition, reducing labeled data dependency by self-supervised learning while using pre-trained action-related models as teacher guidance. Semi-supervised methods that combine labeled and unlabeled data for joint training have attracted increasing attention for action localization tasks, since video annotation is more tedious and time-consuming compared with image annotation.

Efficiency Improvements

Noisy Student Training makes the student larger than, or at least equal to, the teacher so the student can better learn from a larger dataset, and it adds noise to the student so the noised student is forced to learn harder from the pseudo labels. The student model is exposed to noise, such as data augmentation and dropout, during training, which helps it generalize better than the teacher model.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. user (2026-06-08T13:06:11.863Z)

maxContentLength size of 10485760 exceeded

25. assistant (2026-06-08T13:06:15.273Z)

I have enough on self-training. Let me get the timestamp-supervision cost figure from the HTML version and search for temporal data augmentation methods for TAL, which is the last gap.

26. user (2026-06-08T13:06:26.518Z)

Web search results for query: ""timestamp supervision" annotation time minutes per video segmentation 6 times faster Breakfast dataset cost"

Links: [{"title":"Timestamp-Supervised Action Segmentation from the Perspective of

Clustering","url":"https://arxiv.org/pdf/2212.11694"},{"title":"Temporal Action Segmentation: A Comprehensive Overview","url":"https://www.emergentmind.com/topics/temporal-action-segmentation"},{"title":"A Generalized & Robust Framework For Timestamp Supervision in","url":"https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136640276.pdf"},{"title":"A Generalized & Robust Framework For Timestamp Supervision in Temporal Action Segmentation","url":"https://arxiv.org/pdf/2207.10137"},{"title":"Timestamp-Supervised Action Segmentation with Graph Convolutional Networks","url":"https://arxiv.org/pdf/2206.15031"},{"title":"Temporal Action Segmentation from Timestamp Supervision","url":"https://github.com/ZheLi2020/TimestampActionSeg"},{"title":"Robust Action Segmentation from Timestamp Supervision","url":"https://arxiv.org/pdf/2210.06501"},{"title":"A Generalized and Robust Framework for Timestamp Supervision in Temporal Action Segmentation | SpringerLink","url":"https://link.springer.com/chapter/10.1007/978-3-031-19772-7_17"},{"title":"A New Dataset and Approach for Timestamp Supervised Action Segmentation Using Human Object Interaction | IEEE Conference Publication | IEEE Xplore","url":"https://ieeexplore.ieee.org/document/10208480"},{"title":"Action Segmentation ? Temporal Annotation for Robot Learning | Claru","url":"https://claru.ai/glossary/action-segmentation"}]

Based on the search results, I found relevant information about timestamp supervision annotation on the Breakfast dataset. Here are the key findings:

Annotation Time and Cost

A user study with 5 annotators labeling 50 videos of the Breakfast dataset found that Timestamp Supervision requires about two-thirds of the annotation time as full supervision, while SkipTag Supervision requires only one-third. This represents a significant reduction in annotation burden compared to full supervision.

Dataset Information

The Breakfast dataset contains 1712 videos of breakfast-related activities with an average length of around 2.3 minutes, and the frames are annotated with 48 fine-grained action classes.

Performance Results

While the search results don't specifically mention a "6 times faster" annotation speed, they do show that timestamp supervision is significantly more efficient than full annotation. State-of-the-art timestamp supervision halves the gap to full supervision (Acc=64.1% vs. 68.0% on Breakfast), making it a practical alternative to fully annotated data.

The specific metric of "6 times faster" with exact minutes per video that you're looking for wasn't found in these results, though the studies confirm substantial time savings with timestamp supervision compared to full supervision.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. user (2026-06-08T13:06:30.454Z)

Web search results for query: "temporal data augmentation video temporal action localization time warping speed jitter segment mixup boundary"

Links: [{"title":"(PDF) Extending Temporal Data Augmentation for Video Action Recognition","url":"https://www.researchgate.net/publication/365259111_Extending_Temporal_Data_Augmentation_for_Video_Action_Recognition"},{"title":"[2008.05721] Learning Temporally Invariant and Localizable Features via Data Augmentation for Video Recognition","url":"https://ar5iv.labs.arxiv.org/html/2008.05721"},{"title":"[2211.04888] Extending Temporal Data Augmentation for Video Action Recognition","url":"https://arxiv.org/abs/2211.04888"},{"title":"Unsupervised Pre-training for Temporal Action Localization Tasks","url":"https://arxiv.org/pdf/2203.13609"},{"title":"Extending Temporal Data Augmentation

for Video Action Recognition | Springer Nature Link","url":"https://link.springer.com/chapter/10.1007/978-3-031-25825-1_8"},{"title":"Extending Temporal Data Augmentation for Video Action Recognition","url":"https://arxiv.org/pdf/2211.04888"},{"title":"Boundary-Centric Active Learning for Temporal Action Segmentation","url":"https://arxiv.org/html/2604.15173v1"},{"title":"Learning Temporally Invariant and Localizable Features via Data Augmentation for Video Recognition","url":"https://arxiv.org/pdf/2008.05721"}]

Based on the search results, I found several relevant research papers and findings related to temporal data augmentation for video action recognition and temporal action localization:

Key Findings

Temporal data augmentation strategies extend to the temporal dimension for videos to learn temporally invariant or temporally localizable features to cover temporal perturbations or complex actions in videos.

Temporal Augmentation Techniques

RandAugment-T extends the spatial-only framework to the time dimension and presents modifications on cut-and-paste and blend algorithms, such as CutOut, CutMix, MixUp, and CutMixUp, to produce temporally localisable features.

Specific Methods Mentioned

The research includes several novel temporal augmentation approaches:

- VideoReverse, FrameFadeIn, and VideoCutMix are transformations designed specifically for video samples, where VideoReverse turns the video backwards while maintaining semantics and integrity.

- FadeMixUp can be modeled for temporal variations and can learn temporally localizable features without sharp boundary changes, like other cut-and-pasting algorithms.

- A scale-aware sampling strategy adds randomness to the action timescale (how fast an action goes), based on the observation that an action video played at different speeds contains almost identical semantics.

Performance Impact

Involving the time dimension when designing augmentations can be superior to spatial-only variants for video action recognition.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-06-08T13:06:36.527Z)

I now have solid coverage across all dimensions. Let me do one final search to nail the data-efficiency claim for VLM video temporal grounding with very few samples (the strongest "few labels go far" anchor for Option B), and confirm the RL/GRPO data-efficiency angle.

29. user (2026-06-08T13:06:44.367Z)

Web search results for query: "VideoChat-R1 reinforcement fine-tuning temporal grounding data efficient hundreds samples outperform SFT GRPO video LLM"

Links: [{"title":"[2504.06958] VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://arxiv.org/abs/2504.06958"},{"title":"VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://arxiv.org/html/2504.06958v5"},{"title":"[Literature Review] VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://www.themoonlight.io/en/review/videochat-r1-enhancing-spatio-temporal-

perception-via-reinforcement-fine-tuning"},{"title":"VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning | Cool Papers - Immersive Paper Discovery","url":"https://papers.cool/arxiv/2504.06958"},{"title":"[2504.06958v3] VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://arxiv.org/abs/2504.06958v3"},{"title":"VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://arxiv.org/pdf/2504.06958"},{"title":"Paper page - VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://huggingface.co/papers/2504.06958"},{"title":"(PDF) VideoChat-R1: Enhancing Spatio-Temporal Perception via Reinforcement Fine-Tuning","url":"https://www.researchgate.net/publication/390639366_VideoChat-R1_Enhancing_Spatio-Temporal_Perception_via_Reinforcement_Fine-Tuning"},{"title":"TimeLens: Rethinking Video Temporal Grounding with Multimodal LLMs","url":"https://arxiv.org/pdf/2512.14698"}]

Based on the search results, I found comprehensive information about VideoChat-R1:

Overview

VideoChat-R1 proposes Reinforcement Fine-Tuning (RFT) as a data-efficient method to enhance video reasoning on specific tasks without sacrificing original capabilities. Through joint RFT on multiple spatio-temporal perception tasks, the researchers developed VideoChat-R1, a powerful Video MLLM.

Data Efficiency

The experiments reveal that RFT is highly data-efficient for task-specific improvements. Through multi-task RFT on spatio-temporal perception objectives with limited samples, VideoChat-R1 achieves state-of-the-art performance on spatio-temporal perception tasks.

Performance Improvements

VideoChat-R1 achieves state-of-the-art spatio-temporal perception, demonstrating significant improvements in tasks like temporal grounding (+31.8) and object tracking (+31.2), while also improving general QA benchmarks.

GRPO vs. SFT

The paper includes ablation studies comparing GRPO to supervised fine-tuning (SFT), and the results indicate that GRPO outperforms SFT and that chain-of-thought doesn't always lead to performance gains in spatiotemporal perception tasks.

Methodology

GRPO works by generating multiple candidate responses to a given question, scoring each response using the MLLM itself as a reward function, and then encouraging the model to favor the higher-scoring responses. The reward function is crucial and is designed based on the specific video task.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. assistant (2026-06-08T13:06:47.012Z)

I now have comprehensive, well-sourced evidence across all four sub-questions, plus precise grounding in our actual data shapes. Let me compile the structured answer.

31. user (2026-06-08T13:09:19.989Z)

Structured output provided successfully

32. assistant (2026-06-08T13:09:34.607Z)

I've completed the research and delivered the structured answer. Here's a summary of what I produced for the LABEL & DATA REUSE / DATA ENGINEERING dimension:

The core thesis: Our ~128 hand-labeled rallies are worth far more than their raw count if we stop treating them as eval-only CSVs and promote them to a single model-agnostic "labeled-corpus" that transcodes into every consumer (heuristics, TAL nets, and the Option-B VLM).

Grounded in our actual code (`backend/eval/training.py::label_candidates`, `metrics.match_intervals`, `candidate_segments` table, collector bridge/ADR-002), I verified that:
- Our rally schema (`start_time,end_time,ending_reason,sport`) is already ~90% of the ActivityNet/ActionFormer TAL JSON schema (`segment:[start,end]` in seconds) ? confirmed against ActionFormer's actual thumos14 loader.
- ending_reason gives a free second QA task; WASB trajectories attach as an auxiliary modality keyed by the same video_id+frame.

Sourced data-efficiency claims across all four sub-questions:
- (a) Format transcoders ? ActivityNet/ActionFormer JSON schema verified from source.
- (b) Five amplification levers, all guardrail-clean: open-teacher pseudo-labeling/SS-TAL (PLR/ALQA, noisy-student), temporal augmentation (RandAugment-T/FadeMixUp), active learning (uncertainty/boundary-centric beats random), timestamp supervision (~? the time of full labels, near-full accuracy), and RFT/GRPO for Option B (VideoChat-R1 +31.8 with limited samples; VTG-R1 filtered-13K > unfiltered-56K).
- (c) Realistic match-count bands per model class ? and the key insight that our "learned < baseline at n=2" is data starvation, not method failure.
- (d) The reusable JSONL spine with provenance/consent/split-by-match fields that enforce the no-paid-LLM + minors-excluded guardrails at the format level.

Top recommendation: Build the canonical corpus format (it IS the moat), grow n?2?5 matches immediately to unblock honest tuning, amplify for free via augmentation + open-teacher pseudo-labels, route vendor budget by active learning, and design Option B around quality-SFT-cold-start + GRPO (our small clean set is the right asset for RL) ? while deferring implementation per the owner's scaffolding-first directive.

The full structured output with per-option pros/cons, costs, license/privacy notes, sources, and caveats has been submitted.